REACT · COURSE OUTLINE

React JS

The Complete Roadmap

Every important React topic in one clear path: from modern JavaScript and components, through Hooks, routing and data, to performance, testing and deployment.

10 parts • Beginner to Advanced • Mini project in every part

COURSE ROADMAP

10

What We'll Cover

Ten parts, one clear path from first component to deployed app.

1 **Foundations: Modern JS & Tooling** — BEGINNER
Modern JavaScript, async code, Node, npm and Vite setup
~1 week

2 **Core React: JSX, Components & Props** — BEGINNER
JSX, components, props, lists, keys, events, conditional rendering
~1.5 weeks

3 **Styling & UI** — BEGINNER
CSS Modules, Tailwind, responsive layout, animation, accessibility
~1 week

4 **State & Forms** — BEGINNER+
useState in depth, controlled inputs, lifting state, validation
~1 week

5 **Hooks (Lecture Deck)** — INTERMEDIATE
Rules of Hooks, useEffect, useContext, useRef, useMemo, useReducer, custom Hooks
~2 weeks

6 **Routing with React Router** — INTERMEDIATE
React Router, dynamic and nested routes, protected routes, loaders
~1 week

7 **Data Fetching & APIs** — INTERMEDIATE
REST, fetch & axios, CRUD, loading states, TanStack Query, RTK Query
~1.5 weeks

8 **Global State Management** — INTERMEDIATE
Context patterns, Redux Toolkit, Zustand, choosing the right tool
~1.5 weeks

9 **Performance & Advanced Patterns** — ADVANCED
Rendering, memoization, code splitting, error boundaries, patterns
~2 weeks

10 **Auth, Testing & Deployment** — ADVANCED
Auth flow, Vitest & RTL, TypeScript with React, deployment
~2 weeks

THE BIG IDEA

3

Your Learning Path

Three levels. Finish each level's mini projects before climbing to the next.

BEGINNER

Get the language of React

~4.5 weeks

1. **Foundations** — *~1 week*
2. **Core React** — *~1.5 weeks*
3. **Styling & UI** — *~1 week*
4. **State & Forms** — *~1 week*

INTERMEDIATE

Build real, data-driven apps

~6 weeks

5. **Hooks** — *~2 weeks*
6. **Routing** — *~1 week*
7. **Data & APIs** — *~1.5 weeks*
8. **Global State** — *~1.5 weeks*

ADVANCED

Think like a professional

~4 weeks

9. **Performance** — *~2 weeks*
10. **Ship It** — *~2 weeks*

> *Suggested pace only. Move faster where you already know the topic, slower where it feels new. About 14 to 15 weeks in total.*

PART 01 OF 10 · BEGINNER

~1 WEEK

1 Foundations: Modern JS & Tooling

The JavaScript and setup you need before React feels easy.

React is just JavaScript with a few new ideas. Get comfortable with modern JS syntax and the Vite toolchain first, and everything after this part becomes much smoother.

MODERN JAVASCRIPT

- let / const & scope
- Arrow functions
- Destructuring, spread & rest
- Template literals
- ES modules: import / export

ARRAYS & OBJECTS

- map, filter, reduce
- find, some, every
- Optional chaining ?. and ??
- Immutable updates with spread

ASYNC JAVASCRIPT

- Promises in simple words
- async / await & try / catch
- fetch API & JSON
- The event loop, simply

TOOLING & BIG IDEA

- Node, npm & package.json
- Vite project setup
- Declarative UI & Virtual DOM
- Env variables, ESLint, Prettier

```
Setup Sprint

// MINI PROJECT
Vite + React starter

- create app with npm create vite
- clean the boilerplate
- render a list using map()
- fetch JSON, show on screen
- push the repo to GitHub
```

CHECKPOINT - MOVE ON WHEN YOU CAN

Start a Vite project from memory and explain what the Virtual DOM is for.

REMEMBER

React is a library for building UI from data. Same data in, same UI out.

PART 02 OF 10 · BEGINNER

2

Core React: JSX, Components & Props

Think in components: small, reusable pieces of UI.

~1.5 WEEKS

Everything in React is a component. Learn to write JSX, split a screen into components, pass data with props, and render lists and conditions.

JSX

- JSX rules & expressions
- Fragments (<></>)
- className, style & attributes
- Self-closing tags

COMPONENTS

- Function components
- Composition & children
- Import / export patterns
- Folder structure

PROPS

- Passing & destructuring props
- Default values
- One-way data flow
- PropTypes / TypeScript types

RENDERING & EVENTS

- Lists & the key prop
- Conditional rendering
- onClick, onChange, onSubmit
- Synthetic events

```
Product Card Gallery

// MINI PROJECT
Product cards grid

- Card component with props
- map() over a data array
- "Out of stock" badge
- onClick logs the product
- children for custom footer
```

CHECKPOINT - MOVE ON WHEN YOU CAN

Break any screen into components and say which props each one needs.

REMEMBER

Props flow DOWN, events flow UP. Data in, UI out.

PART 03 OF 10 · BEGINNER

~1 WEEK

3 Styling & UI

Make it look good and work on every screen.

Pick a styling approach and stick with it. This part covers CSS Modules, Tailwind, responsive layout, animation and the accessibility basics every interface needs.

STYLING OPTIONS

- CSS & CSS Modules
- Tailwind CSS utilities
- Inline styles (when it is ok)
- styled-components overview

RESPONSIVE DESIGN

- Mobile-first thinking
- Flexbox & Grid
- Breakpoints
- Relative units (rem, %, vh)

UI POLISH

- Dark mode with CSS variables
- Icons: lucide / react-icons
- shadcn/ui & MUI overview
- Animation with framer-motion

ACCESSIBILITY

- Semantic HTML
- Labels & alt text
- Keyboard focus
- Colour contrast

```
Responsive Landing Page

// MINI PROJECT
Landing page, 3 breakpoints

- navbar with mobile menu
- hero + feature grid
- dark / light toggle
- fade-in on scroll
- passes keyboard-only test
```

CHECKPOINT - MOVE ON WHEN YOU CAN

Build one section that looks right at phone, tablet and desktop widths.

REMEMBER

Define colours and spacing once as tokens, then reuse them everywhere.

PART 04 OF 10 · BEGINNER+

4 State & Forms

Make the UI respond to the user.

~1 WEEK

State is data that changes over time. Learn to store it correctly, share it between components, and build forms that validate input without fighting React.

useState IN DEPTH

- Updater form: setX(prev => ...)
- Objects & arrays, immutably
- Batching & re-render rules
- Lazy initial state

FORMS

- Controlled inputs
- Uncontrolled inputs + refs
- Checkbox, radio, select
- onSubmit & preventDefault

STRUCTURING STATE

- Lifting state up
- Derived values: do not store
- One source of truth
- Reset state with a key

VALIDATION

- Manual validation
- React Hook Form
- Schema validation (Zod / Yup)
- Error & loading UI

```
Signup Form + Todo App

// MINI PROJECT
Signup form with validation

- controlled name / email / pass
- inline error messages
- disable submit while invalid
- Todo list: add, toggle, delete
- filter: all / active / done
```

CHECKPOINT - MOVE ON WHEN YOU CAN

Explain controlled vs uncontrolled inputs and why we never mutate state.

REMEMBER

If you can calculate it from existing state, do not store it as state.

PART 05 OF 10 · INTERMEDIATE

5

Hooks (Lecture Deck)

Give function components memory, effects and reusable logic.

COVERED IN YOUR PPT

Hooks let function components remember things and react to changes without classes. The eight core Hooks are already covered slide by slide in your Hooks lecture deck.

RULES & BASICS

- The two golden rules
- What a Hook really is
- useState recap
- Call order & re-renders

EFFECTS & REFS

- useEffect & dependency array
- Cleanup functions
- useRef & DOM access
- useLayoutEffect (rare)

CONTEXT & PERFORMANCE

- useContext & Provider
- useMemo
- useCallback + React.memo
- When NOT to memoize

ADVANCED & MODERN

- useReducer
- Custom Hooks
- useId, useTransition
- React 19: use, useActionState

Custom Hook Library

```
// MINI PROJECT
Your own hooks folder

- useLocalStorage(key, initial)
- useDebounce(value, delay)
- useFetch(url)
- useWindowWidth()
- demo page using all four
```

CHECKPOINT - MOVE ON WHEN YOU CAN

Say when each of the 8 core Hooks is the right tool, without notes.

REMEMBER

Custom Hooks share LOGIC, not state. Each caller gets its own copy.

PART 06 OF 10 · INTERMEDIATE

6

Routing with React Router

Turn one page into a full multi-page app.

~1 WEEK

Routing maps URLs to components. Learn static and dynamic routes, nested layouts, navigation helpers and how to guard pages that need a login.

ROUTER BASICS

- createBrowserRouter
- Link & NavLink
- Route & element
- 404 / catch-all route

DYNAMIC & NESTED

- URL params (:id) & useParams
- Nested routes & Outlet
- Index & layout routes
- Shared navbar layout

NAVIGATION TOOLS

- useNavigate
- Search params & filters
- Redirects with Navigate
- Location state

ADVANCED ROUTING

- Protected routes
- Lazy-loaded routes
- Loaders & actions
- Error elements

Multi-page Blog

```
// MINI PROJECT
Blog with 5 routes

- / , /posts , /posts/:id
- shared layout with Outlet
- search term in the URL
- /admin is protected
- custom 404 page
```

CHECKPOINT - MOVE ON WHEN YOU CAN

Build nested routes with a dynamic :id and read it with useParams.

REMEMBER

The URL is state too. Let it decide what the page shows.

PART 07 OF 10 · INTERMEDIATE

7

Data Fetching & APIs

Connect your UI to real data.

~1.5 WEEKS

Most apps live on server data. Learn REST basics, fetch and axios, the loading / error / empty states, full CRUD, and modern caching tools like TanStack Query and RTK Query.

HTTP & REST

- GET, POST, PUT, PATCH, DELETE
- Status codes
- Headers & JSON bodies
- Test with JSONPlaceholder

FETCHING BASICS

- fetch vs axios
- useEffect + fetch pattern
- Loading, error & empty states
- AbortController & race bugs

CRUD IN REACT

- Create, read, update, delete UI
- Optimistic updates
- Axios instance & interceptors
- Pagination & search

SERVER STATE TOOLS

- TanStack Query basics
- Caching & refetching
- Mutations & invalidation
- RTK Query overview

```
Movie Search App

// MINI PROJECT
Search + details app

- debounced search box
- loading skeletons + errors
- paginated result cards
- details route /movie/:id
- favourites saved to cache
```

CHECKPOINT - MOVE ON WHEN YOU CAN

Do full CRUD against an API and handle loading and error states cleanly.

REMEMBER

Server data is a cache, not state you own. Let a library manage it.

PART 08 OF 10 · INTERMEDIATE

8

Global State Management

Share state across the whole app without chaos.

~1.5 WEEKS

When many distant components need the same data, props get painful. Learn Context patterns first, then Redux Toolkit and a lightweight store like Zustand, and how to choose.

WHEN TO GO GLOBAL

- Prop drilling pain
- Local vs shared state
- Server vs client state
- Choosing a tool

CONTEXT PATTERNS

- Provider + custom hook
- Splitting contexts
- Context + useReducer
- Re-render pitfalls

REDUX TOOLKIT

- Store & slices
- Actions & reducers
- createAsyncThunk
- Selectors & DevTools

LIGHTWEIGHT OPTIONS

- Zustand basics
- Jotai idea (atoms)
- Persisting state
- Middleware overview

Shopping Cart

```
// MINI PROJECT
Cart shared app-wide

- add / remove / quantity
- cart badge in navbar
- totals via selectors
- persist to localStorage
- build once in Redux, once Zustand
```

CHECKPOINT - MOVE ON WHEN YOU CAN

Explain why you chose Context or a store for a given feature.

REMEMBER

Reach for the simplest tool: props, then Context, then a store.

PART 09 OF 10 · ADVANCED

9

Performance & Advanced Patterns

Understand how React renders, then make it fast.

~2 WEEKS

Fast apps come from understanding rendering, not from sprinkling memo everywhere. Learn what triggers renders, how to measure, how to split code, and the patterns pros use.

HOW REACT RENDERS

- Reconciliation & keys
- Render vs commit phase
- Strict Mode double render
- Automatic batching

OPTIMISATION

- React.memo
- useMemo / useCallback
- List virtualisation
- Profiler & DevTools

LOADING STRATEGY

- React.lazy code splitting
- Suspense boundaries
- Transitions
- Image & bundle size

PATTERNS

- Error boundaries
- Portals & modals
- Compound components
- HOC, render props, forwardRef

Dashboard Speed-up

```
// MINI PROJECT
Slow dashboard, fixed

- profile with React DevTools
- virtualise a 10,000-row list
- lazy-load heavy charts
- add an error boundary
- compare bundle size before/after
```

CHECKPOINT - MOVE ON WHEN YOU CAN

Find a real slow render with the Profiler and fix it with evidence.

REMEMBER

Measure first, optimise second. The Profiler beats guessing.

PART 10 OF 10 · ADVANCED

10

Auth, Testing & Deployment

Make it secure, tested and live on the internet.

~2 WEEKS

Real apps need logins, tests and a public URL. Learn the JWT auth flow, test with Vitest and React Testing Library, add TypeScript, and deploy with confidence.

AUTHENTICATION

- Login / signup flow
- JWT & cookies
- Protected & role-based routes
- Token refresh idea

TESTING

- Vitest / Jest
- React Testing Library
- Mock APIs with MSW
- Test behaviour, not internals

TYPESCRIPT + REACT

- Typing props & state
- Typing hooks & events
- Generic components
- tsconfig basics

SHIP IT

- Production build
- Env vars & secrets
- Deploy: Vercel / Netlify
- Next steps: Next.js, RSC

```
Auth Dashboard, Deployed

// CAPSTONE
Role-based dashboard

- signup, login, logout
- admin vs user pages
- 10+ tests passing
- written in TypeScript
- live URL in your portfolio
```

CHECKPOINT - MOVE ON WHEN YOU CAN

Ship an app with auth and tests to a live URL.

REMEMBER

Test what the user sees and does, not how the code is written.

P

PRACTICE MAKES PERMANENT

The Project Ladder

Reading is not enough. Build at least one project from every row.

BEGINNER

- Counter + Todo list
- Quote / weather widget
- Personal portfolio site
- Responsive landing page

INTERMEDIATE

- Movie search with API
- Multi-page blog with routing
- Shopping cart (Redux or Zustand)
- Notes app with localStorage

ADVANCED

- Auth dashboard with roles
- Kanban board with drag and drop
- Chat app with live updates
- Full-stack MERN app, deployed

S

HOW YOUR NOTES ARE ORGANISED

The PDF Series

One part at a time, in the same slide format as your Hooks lecture.

FILE ORDER

1. `React_Part01_Foundations.pdf`
2. `React_Part02_Core_React.pdf`
3. `React_Part03_Styling_UI.pdf`
4. `React_Part04_State_Forms.pdf`
5. `React_Part05_Hooks.pdf` — DONE
6. `React_Part06_Routing.pdf`
7. `React_Part07_Data_APIs.pdf`
8. `React_Part08_Global_State.pdf`
9. `React_Part09_Performance.pdf`
10. `React_Part10_Ship_It.pdf`

Every part PDF has

```
01  Title slide
02  Roadmap of the part
03  The big idea
04  Rules / before you start
05+ One topic per slide:
     explanation, code,
     key points, memory tip
NN  Cheat sheet + Thank you
```

NEXT UP

Part 5 (Hooks) is already finished as your lecture deck. Next in the series: Part 1, then onward in order.



Let's Build

Understand it. Build it. Break it. Fix it. Repeat.

Foundations · Core React · Styling · State · Hooks · Routing · APIs · Global State · Performance · Ship It